Distributed Event Processing Pipeline

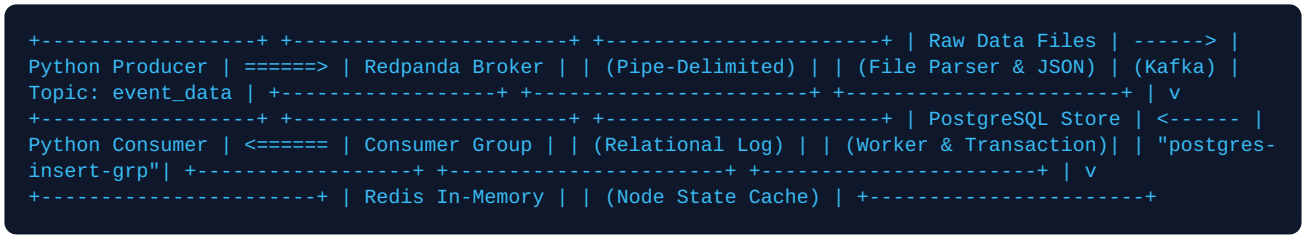
Technical Architecture, Component Specifications, and Local Execution & Testing Guide

Status: Production Ready (Local Dev Profile) **Core Stack:** Redpanda, Python 3, PostgreSQL, Redis **Target Environment:** Standalone Docker & Local Daemon

1. Executive Summary & Architecture Overview

This design document outlines a robust, high-throughput event streaming and dual-persistence pipeline. Designed to eliminate cloud networking friction during development and interviews, the architecture operates locally via Docker Compose while maintaining enterprise-grade design patterns. Raw telemetry is ingested from flat files, structured into JSON payloads, streamed through a high-performance Redpanda (Kafka-compatible) broker, and concurrently consumed to update both an operational Redis state registry and a relational PostgreSQL event store.

End-to-End Data Flow Topology



2. Core Infrastructure & Services Specification

Service Name	Docker Image	Container Port	Host Port	Primary Role & Configuration
Redpanda	`docker.redpanda.com/...`	9092 / 29092	9092 / 29092	Kafka-compatible event streaming broker. Configured for single SMP core, 1GB memory cap, and plaintext external listener.
PostgreSQL	`postgres:15-alpine`	5432	5432	Relational event storage. Automatically provisions `event_store` database with persistent volume backing.
Redis	`redis:7-alpine`	6379	6379	In-memory data store for real-time node registry tracking, metrics caching, and event counting pipelines.

3. Component Architecture & Code Implementation

Producer & Parser Module

Reads raw pipe-delimited telemetry files (`test_data.txt`), validates fields, serializes into JSON, and pushes payloads to Redpanda.

Consumer Worker Engine

Polls messages from Redpanda utilizing consumer group isolation (`postgres-insert-group`), writing concurrently to PostgreSQL and Redis.

```
def raw_to_dict(raw_data):
    raw_list = raw_data.strip("\n").split("|")
    if len([x for x in raw_list if not x]) > 0:
        return None
    return {
        "event_id": raw_list[0],
        "node_id": raw_list[1],
        "event_type": raw_list[2],
        "metric_value": raw_list[3]
    }
```

```
while True:
    event_data = km.poll_single_message()
    if event_data is None:
        continue
    data_tuple = (
        event_data["event_id"],
        event_data["node_id"],
        event_data["event_type"],
        event_data["metric_value"]
    )
    db.insert_table(query, data_tuple)
    rm.update_node_registry(event_data)
```

4. Pipeline Testing & Verification Guide

To validate end-to-end ingestion, message delivery, and real-time state synchronization across both relational and in-memory stores, follow the verification procedures below.

Step 1: Infrastructure Deployment & Health Check

Spin up the containerized stack and verify all services are active and listening on their respective host ports:

```
docker-compose up -d
docker ps  # Verify redpanda_broker (9092), postgres_db (5432), and redis_cache (6379)
```

Step 2: Produce Test Telemetry

Stream the pipe-delimited sample records from `test_data.txt` through the producer into the Redpanda broker:

```
python3 producer.py test_data.txt
```

Step 3: Verify Consumer Execution & PostgreSQL Persistence

Run the consumer daemon to process events from the queue, commit offsets, and persist records to the relational store:

```
python3 consumer.py
```

Verify records in PostgreSQL by executing a query inside the container:

```
docker exec -it postgres_db psql -U postgres -d event_store -c "SELECT COUNT(*), event_type FROM events_table
GROUP BY event_type;"
```

Step 4: Verify Redis Real-Time State & Node Registry

The consumer updates Redis using atomic hashes and sets via `RedisManager`. You can inspect node telemetry and event counts directly via the Redis CLI:

```
# Check all registered node IDs in the global tracking set
docker exec -it redis_cache redis-cli SMEMBERS global_nodes_registry

# Inspect real-time telemetry and aggregated event count for a specific node
docker exec -it redis_cache redis-cli HGETALL node:workstation-atl-01
```

Expected Redis Hash output includes `last_event_id`, `event_type`, `metric_value`, `last_seen` timestamp, and incremented `event_count`.